

IPC: Pipe e FIFO

Contiene lucidi tratti da: 2006-2007 Marco Faella, Clemente Galdi, Giovanni Schmid (Università di Napoli Federico II), 2005-2007 Francesco Pedullà, Massimo Verola (Uniroma2), 2001-2005 Renzo Davoli (Università di Bologna), Alberto Montresor (Università di Bologna).

Interprocess Communication (IPC)

- Per cooperare, i processi hanno bisogno di comunicare
- I segnali sono un primo modo di farlo
 - Il messaggio consiste nel “numero del segnale” ed, eventualmente, le informazioni in `siginfo_t`.
- Vogliamo però trasmettere informazioni arbitrarie

Le pipe

Le **pipe** forniscono un meccanismo attraverso il quale l'**output** di un processo **diviene l'input** per un altro processo.

Il più semplice esempio di tale meccanismo è fornita dall'operatore di pipe **|** di una shell:

ls | grep old

è una pipeline grazie alla quale l'**output** del comando **ls** viene passato in **input** al comando **grep**, **senza** un **file** intermedio **di appoggio**.

La comunicazione tra i due comandi è **unidirezionale** (da **ls** a **grep**, il viceversa non è possibile), e la **sincronizzazione** è ottenuta arrestando **grep** quando non c'è nulla da leggere e arrestando **ls** quando la pipe è piena.

Le pipe

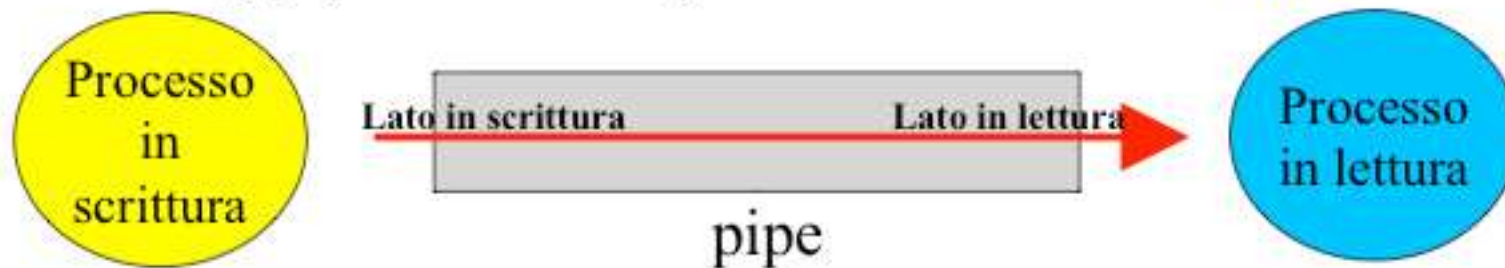
- Le pipe (*tubi*) sono canali di comunicazione a senso unico tra due processi
- I processi devono essere imparentati
 - tipicamente, un padre e un figlio
- Un processo scrive sulla pipe (usando write)
- Un altro processo legge dalla stessa pipe (usando read)

Le pipe

- **Cos'è un pipe?**
 - E' un canale di comunicazione che unisce due processi
- **Caratteristiche:**
 - La più vecchia e la più usata forma di *interprocess communication* utilizzata in Unix
 - Limitazioni
 - Sono half-duplex (comunicazione in un solo senso)
 - Utilizzabili solo tra processi con un "antenato" in comune
 - Come superare queste limitazioni?
 - Gli *stream pipe* sono full-duplex
 - *FIFO (named pipe)* possono essere utilizzati tra più processi
 - *named stream pipe* = stream pipe + FIFO

Le pipe

Le pipe tra processi, realizzate attraverso la **chiamata di sistema pipe** o la funzione **popen** della **libreria STDIO**, sono del tutto analoghe: i dati immessi da un processo nella pipe con successive scritture sono accodati nell'attesa che un altro processo le legga; la coda è gestita con criterio **FIFO**.



Una pipe ha una **dimensione finita**, il cui valore è definito dalla costante **PIPE_BUF**: soltanto un numero massimo di byte può essere scritto o letto ogni volta.

Una pipe è subordinata all'organizzazione gerarchica dei processi: affinché due processi possano comunicare in pipeline è necessario un **antenato comune** che abbia predisposto una pipe a questo fine.

La funzione Pipe

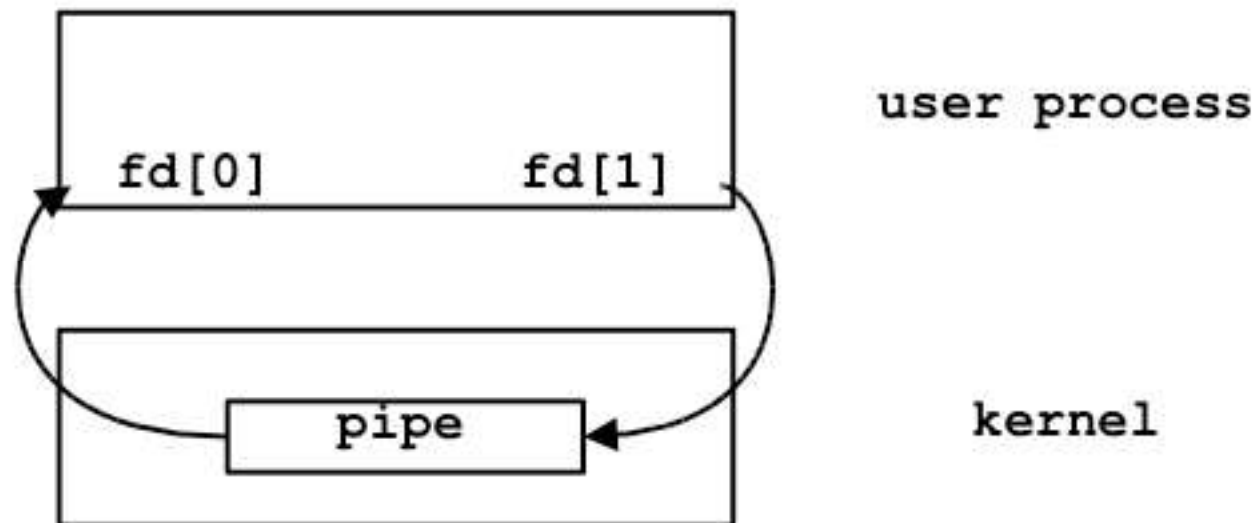
```
#include <unistd.h>

int pipe ( int filedes[2] );
```

- l'argomento *filedes* è costituito da due descrittori di file:
 - *filedes[0]* è aperto in lettura e rappresenta il lato in lettura della pipe ;
 - *filedes[1]* è aperto in scrittura e rappresenta il lato in scrittura della pipe;
 - l'output di *filedes[1]* è l'input per *filedes[0]*;
- restituisce 0 in caso di successo, -1 altrimenti.

Le pipe

- **System call:** `int pipe(int filedes[2]);`
 - Ritorna due descrittori di file attraverso l'argomento **filedes**
 - **filedes[0]** è aperto in lettura
 - **filedes[1]** è aperto in scrittura
 - L'output di **filedes[1]** (estremo di write del pipe) è l'input di **filedes[0]** (estremo di read del pipe)



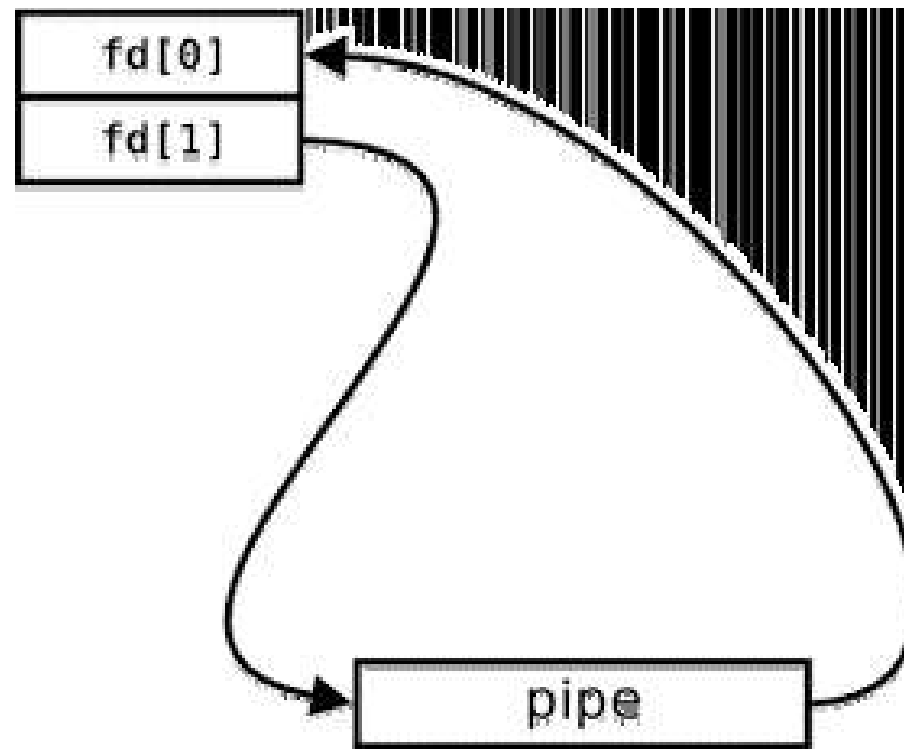
Le pipe

- Consideriamo il frammento:

```
int fd[2];  
  
if (pipe(fd) < 0)  
    perror("pipe"), exit(1);
```

- dopo la sua esecuzione:
 - fd[0] e' il descrittore per **leggere** dalla pipe
 - fd[1] e' il descrittore per **scrivere** sulla pipe

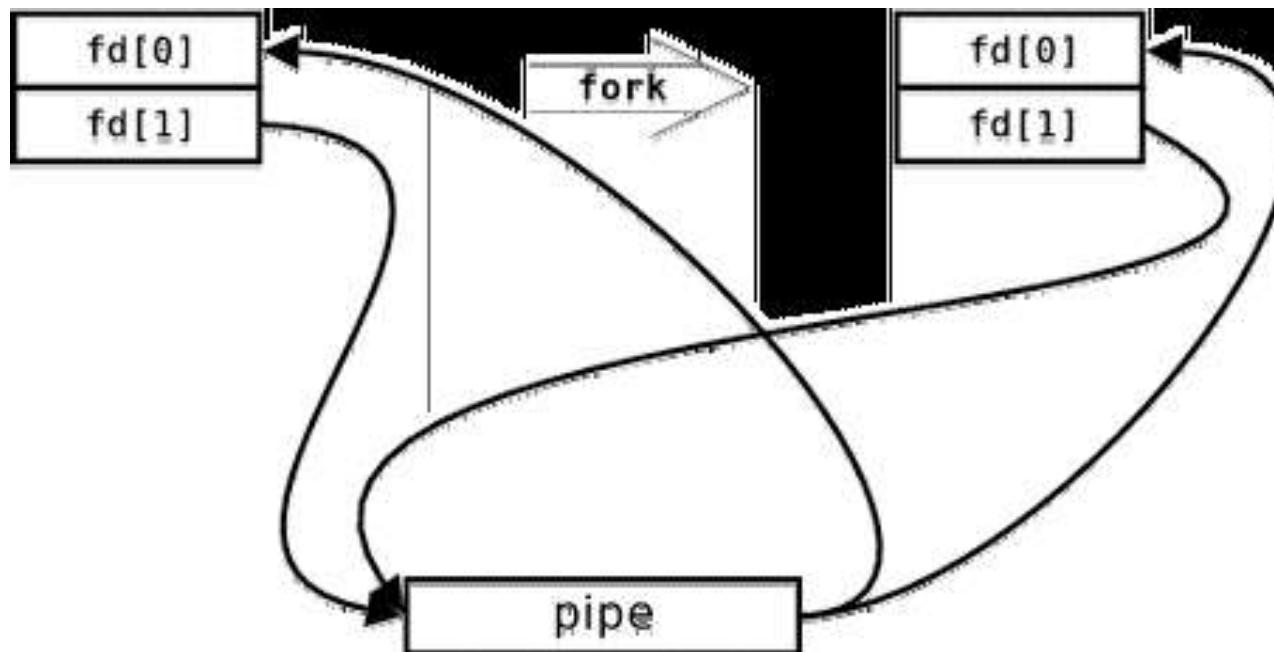
Le pipe



Generata da un singolo processo ha poca utilità ...

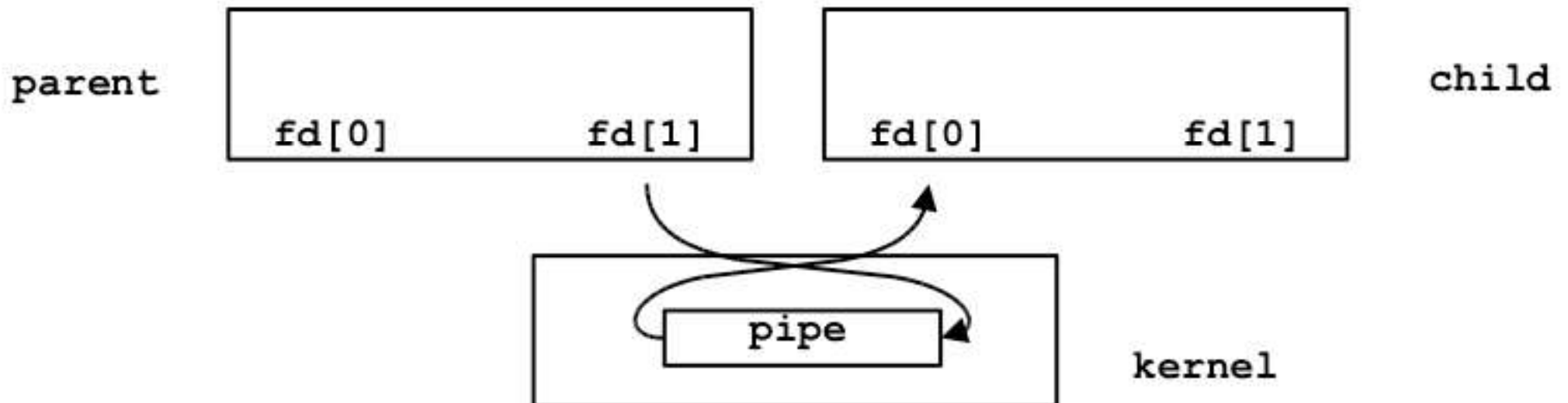
Utilizzo pipe

- Tipicamente, un processo crea una pipe e poi chiama fork



Utilizzo pipe

- **Come utilizzare i pipe?**
 - Cosa succede dopo la `fork` dipende dalla direzione dei dati
 - I canali non utilizzati vanno chiusi
- **Esempio: parent → child**
 - Il parent chiude l'estremo di read (`close(fd[0]);`)
 - Il child chiude l'estremo di write (`close(fd[1]);`)



Utilizzo pipe: read

- **Come utilizzare i pipe?**
 - Una volta creati, è possibile utilizzare le normali chiamate **read/write** sugli estremi
- **La chiamata read**
 - se l'estremo di write è aperto
 - restituisce i dati disponibili, ritornando il numero di byte
 - successive chiamate si bloccano fino a quando nuovi dati non saranno disponibili
 - se l'estremo di write è stato chiuso
 - restituisce i dati disponibili, ritornando il numero di byte
 - successive chiamate ritornano 0, per indicare la fine del file

Utilizzo pipe: write

- **La chiamata write**

- se l'estremo di read è aperto
 - i dati in scrittura vengono bufferizzati fino a quando non saranno letti dall'altro processo
- se l'estremo di read è stato chiuso
 - viene generato un segnale **SIGPIPE**
 - ignorato/catturato: write ritorna **-1** e **errno=EPIPE**
 - azione di default: terminazione

- **Esercizio:**

- Due processi: parent e child
- Il processo parent comunica al figlio una stringa, e questi provvede a stamparla

Una pipe tra padre e figlio

```
int fd[2];

if (pipe(fd) < 0)
    perror("pipe"), exit(1);
if ( (pid=fork()) < 0 )
    perror("fork"), exit(1);
else if (pid>0) { // padre
    close(fd[0]);
    write(fd[1], "ciao!", 5);
} else { // figlio
    close(fd[1]);
    n = read(fd[0], buf, sizeof(buf));
    write(STDOUT_FILENO, buf, n);
}
```

```
/* pipel: invio di dati da un genitore ad un figlio */
```

```
#include <stdio.h>
#include <unistd.h>
#define MAXLINE 64
```

```
int main(void)
{
    int n, fd[2];
    pid_t pid;
    char line[MAXLINE];

    if (pipe(fd) < 0) perror("pipe"), exit(1);

    if ( (pid = fork()) < 0) perror("fork"), exit(1);

    else if (pid > 0) { /* genitore */
        close(fd[0]);
        write(fd[1], "hello world\n", 12);
    }
    else { /* figlio */
        close(fd[1]);
        n = read(fd[0], line, MAXLINE);
        write(STDOUT_FILENO, line, n);
    }
    exit(0);
}
```

Leggere e scrivere sulle pipe

- All'inizio una pipe è vuota
- write aggiunge dati alla pipe
- read legge *e rimuove* dati dalla pipe
 - non si possono leggere piu' volte gli stessi dati da una pipe
 - non si puo' chiamare lseek su una pipe
 - i dati si ottengono in ordine First In First Out
- una pipe con una estremita' chiusa si dice rotta (broken)

Leggere e scrivere sulle pipe

- Scrivere: write aggiunge i suoi dati alla pipe
 - se la pipe e' rotta, viene generato il segnale SIGPIPE e write restituisce un errore
- Leggere: read(fd[0], buf, 100)
 - meno di 100 bytes nella pipe: read legge l'intero contenuto della pipe
 - piu' di 100 bytes nella pipe: read legge i primi 100 bytes
 - pipe vuota: read si blocca in attesa di dati
 - pipe vuota e rotta: read restituisce 0